FOR COMPLETE BEGINNERS

Extends can break rules.

Complete Modern Java, episode 27: composition or inheritance, in Java and TypeScript

> Is it a kind of, or does it have one? A cart has products; it is not a product, and it is not a list.

Code can compile and still model the wrong thing: CartAsProduct extends Product compiles without a complaint.

A fresh product is a product: inheritance. A cart has products: a field. That is composition.

In this episode

- is-a or has-a
- Composition
- Extending a list hands out every list method
- The cart chooses its operations
- TypeScript private at run time

Coming next

- Abstract classes (episode 28)

At the time of recording: Java 25, Node 24 LTS, IntelliJ IDEA 2026.1.3.

01 What each run printed

javac CartAsProduct.java	`no complaint`
cart: Rice and fresh Milk; count, total	`2, 298`
trolley: add Rice, add null; size	`2`
trolley.clear(); size	`0`
cart.add(null) / cart.clear()	`exception / compile error`
carts.ts: cart.items.length = 0	`TS2341, then node prints 0`

Every row was run, not predicted.

02 The tempting shortcut

Trolley inherits every ArrayList method, so add(null) and clear() just work, and the cart's rule is gone.

Extending a list is not always wrong; it hands out every operation the list has, including ones you never chose.

Trolley.java and Carts.java

```
class Trolley extends ArrayList<Product> {}

Trolley trolley = new Trolley();
trolley.add(new Product("Rice", 199));
trolley.add(null);
IO.println(trolley.size());   // 2
trolley.clear();
IO.println(trolley.size());   // 0
```

> Printed 2, then 0.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

03 The cart has a list

ShoppingCart keeps its list private and decides which operations exist. Want clear later? Add it, on purpose.

ShoppingCart.java (from episode 20)

```
class ShoppingCart {
    private ArrayList<Product> items = new ArrayList<>();

    void add(Product product) {
        if (product == null) {
            throw new IllegalArgumentException("product cannot be null");
        }
        items.add(product);
    }
    // count, totalCents, remove, removeByName
}
```

> cart.add(null): product cannot be null. cart.clear(): cannot find symbol.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

04 TypeScript private has a catch

tsc refuses cart.items, but private is a checker rule: node runs the line and the count becomes 0.

carts.ts

```
class ShoppingCart {
    private items: Product[] = [];
    // add, count, totalCents
}
cart.items.length = 0;      // TS2341 from tsc
console.log(cart.count());  // node prints 0
```

> tsc: Property items is private and only accessible within class ShoppingCart. node: 0.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

Is it, or does it have?

1. Really a more specific kind? Inheritance may fit.
2. Has or uses another? A field: composition.
3. Unsure? The field is usually the simpler start.
4. Compiling is not the same as modelling well.
5. Challenge: fresh product, cart, delivery driver.

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The code and the challenge files: complete-modern-java
https://github.com/code-with-sam-dev/complete-modern-java

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. TypeScript is a trademark of Microsoft. IntelliJ IDEA is a trademark of JetBrains. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-11-07 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.